

EN UNA FRASE Dejas de conectar tus clases con `new`: las anotas como beans, Spring las crea y las inyecta por constructor, y el proyecto queda organizado en capas (controlador, servicio, repositorio, dominio).

LO ESENCIAL

Inversión de Control (IoC)	El framework crea y conecta los objetos; tú declaras qué necesitas. Evita el acoplamiento del <code>new</code>
Contenedor IoC / Application Context	Núcleo de Spring: instancia los beans, resuelve sus dependencias y gestiona su ciclo de vida
Bean	Objeto cuyo ciclo de vida gestiona Spring
@Component	Bean genérico; base de las demás anotaciones estereotipo
@Service	Bean de lógica de negocio: reglas y orquestación de repositorios
@Repository	Bean de acceso a datos
@RestController	Bean de presentación: recibe peticiones HTTP y devuelve JSON
Inyección por constructor	La recomendada: campo <code>final</code> , dependencias explícitas, testeable sin Spring. Con un único constructor no hace falta <code>@Autowired</code>
@Autowired en campo o setter	Alternativas más débiles: el campo no puede ser <code>final</code> ; el setter solo tiene sentido si la dependencia es opcional
@PostConstruct	Método que Spring llama una vez, tras crear e inyectar el bean; aquí se cargan los datos iniciales
@PreDestroy	Método que Spring llama antes de destruir el bean (no se llama en beans prototype)
Ámbito singleton	Una instancia por contenedor; es el valor por defecto
@Scope("prototype")	Una instancia nueva en cada inyección
Libro como clase, no record	JPA (Capítulo 9) exige constructor sin argumentos y setters

ERRORES TÍPICOS

1 The dependencies of some of the beans ... form a cycle

- ✓ A necesita B y B necesita A. Rompe el ciclo extrayendo la responsabilidad compartida a un tercer bean del que dependen ambos.

2 required a single bean, but 2 were found

- ✓ hay dos beans del mismo tipo. Marca el que va por defecto con `@Primary` o elige uno con `@Qualifier("nombre")` en el parámetro del constructor.

3 required a bean of type 'BibliotecaService' that could not be found

- ✓ a la clase le falta su anotación estereotipo. Añade `@Service`, `@Repository` o `@RestController` según la capa.

4 El mismo mensaje con la anotación puesta

- ✓ la clase está fuera del alcance de `@ComponentScan`. Llévala a un subpaquete de `com.javaesencial.biblioteca` o configura el escaneo con `@ComponentScan("com.externa.servicios")`.

5 El método @PreDestroy de un bean prototype nunca se llama

- ✓ Spring crea los beans prototype, pero no los destruye. Libera sus recursos a mano.

EN EL PROYECTO BIBLIOTECA

El proyecto se reorganiza en cuatro paquetes. Son nuevos `dominio/Libro`, `repositorio/RepositorioLibros` (`@Repository` con tres libros iniciales, `findAll` y `findById`), `servicio/BibliotecaService` (`@Service`) y `controlador/LibroController`, que expone `GET /libros`. `HolaMundoController` y `LibroRespuesta` se mueven al paquete `controlador` sin cambiar su código. El flujo de una petición queda así: `LibroController` → `BibliotecaService` → `RepositorioLibros`, y Jackson serializa la lista a JSON.

■ CÓDIGO MÍNIMO Y PETICIONES

El repositorio en memoria, con los datos iniciales cargados en `@PostConstruct`:

```
@Repository
public class RepositorioLibros {

    private final List<Libro> libros = new ArrayList<>();

    @PostConstruct
    public void cargarDatosIniciales() {
        libros.add(new Libro(1L, "El Quijote", "Miguel de Cervantes", 1605));
        libros.add(new Libro(2L, "1984", "George Orwell", 1949));
        libros.add(new Libro(3L, "Dune", "Frank Herbert", 1965));
    }

    public List<Libro> findAll() {
        return libros;
    }
}
```

JAVA

El servicio recibe el repositorio por constructor, sin `@Autowired` y sin `new`:

```
@Service
public class BibliotecaService {

    private final RepositorioLibros repositorio;

    public BibliotecaService(RepositorioLibros repositorio) {
        this.repositorio = repositorio;
    }

    public List<Libro> obtenerTodos() {
        return repositorio.findAll();
    }
}
```

JAVA

El controlador recibe el servicio de la misma forma y expone `GET /libros`:

```
@RestController
public class LibroController {

    private final BibliotecaService service;

    public LibroController(BibliotecaService service) {
        this.service = service;
    }

    @GetMapping("/libros")
    public List<Libro> listarTodos() {
        return service.obtenerTodos();
    }
}
```

JAVA